Music Genre Classification on GTZAN

RW
School of ECS
University of S
.ac.uk

Abstract

This report presents a comparative study of six neural classifiers for GTZAN music genre classification. The first four models classify 180×180 *Mel-spectrogram images using a fully connected network, a CNN, a CNN with batch normalisation, and the same batch-normalised CNN trained with RMSProp. The final two models use audio-derived feature sequences with an LSTM classifier, with and without conditional GAN augmentation. A fixed 70/20/10 train/validation/test split is used, validation accuracy selects the best checkpoint, and the test set is used only for final reporting. The best model is the batch-normalised CNN trained for 100 epochs, achieving 75.2% test accuracy. The best audio model is the LSTM, achieving 72.0% test accuracy. The results show that architecture and input representation are more important than optimiser choice, and that GAN augmentation is useful only when generated samples are sufficiently realistic.*

1. Introduction

This coursework addresses the GTZAN ten-class music genre classification task. Six neural network structures are implemented and compared: a fully connected network, a CNN, a CNN with batch normalisation, the same CNN trained with RMSProp, an LSTM audio classifier, and an LSTM classifier trained with GAN-based data augmentation. The goal is to evaluate how architecture, batch normalisation, optimiser choice, temporal modelling, and GAN augmentation affect music classification performance.

2. Dataset and Pre-processing

The GTZAN dataset contains ten genres: blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, and rock. It is small and approximately class-balanced, with about 100 tracks per genre, while several genres share timbral or rhythmic traits. This makes the task sensitive to the chosen split and motivates class-level error analysis rather than relying only on overall accuracy. For `Net1`–`Net4`, the provided Mel-spectrogram images are used. Each image is resized to 180×180 using `torchvision.transforms.Resize`, converted to a tensor, and normalised. For `Net5` and `Net6`, audio-derived 3-second feature sequences are used: each 30-second track becomes a sequence of ten audio feature vectors.

All experiments use a fixed random seed and a class-balanced 70/20/10 split for training, validation, and testing. The same split, preprocessing pipeline, and evaluation protocol are used across comparable models. The validation set is used for model selection and tuning, while the test set is only used for final evaluation. Audio features are standardised using training-set statistics only, so validation and test data do not leak into preprocessing. The code automatically uses CUDA if available, otherwise CPU.

3. Model Architectures and Training Methods

3.1. Architectures

Table 1 summarises the architectures. `Net1` is a fully connected baseline with two hidden layers of 512 and 128 units. `Net2` follows the required CNN pattern: Conv-Conv-Pool-Conv-Conv-Pool-FC, using 3×3 kernels and channels 32, 64, 128, and 128. `Net3` adds batch normalisation after convolutional layers. `Net4` keeps the `Net3` architecture but uses RMSProp. `Net5` is a two-layer bidirectional LSTM for audio sequences. `Net6` uses the same LSTM classifier, but augments only the training set with a conditional GAN that generates genre-conditioned audio feature sequences. The trainable parameter counts are approximately 49.8M for `Net1`, 0.768M for `Net2`, 0.769M for `Net3`/`Net4`, and 0.180M for `Net5`/`Net6`; thus `Net1` is simple conceptually but very large because of the flattened image input, showing why parameter count alone does not imply better inductive bias.

3.2. Training Details

All classifiers use cross-entropy loss. The submitted runs use batch size 16. `Net1`, `Net2`, and `Net3` use Adam with

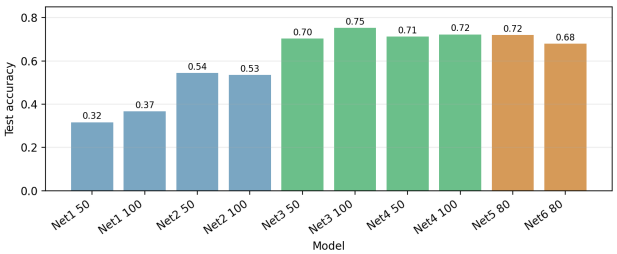

Figure 1. Test accuracy comparison across the final submitted experiments.

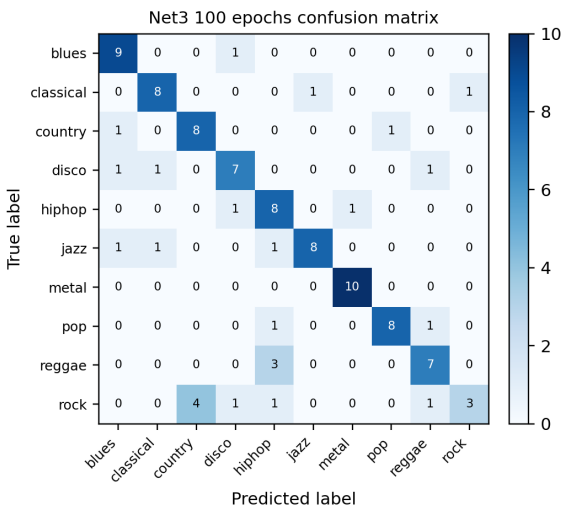

Figure 2. Confusion matrix for the best model, `Net3` trained for 100 epochs.

learning rate 10^{-4}. `Net4` uses RMSProp with learning rate 10^{-4}. The four image models are trained for both 50 and 100 epochs. `Net5` and `Net6` are trained for 80 epochs using Adam with learning rate 10^{-4}. The model checkpoint with the best validation accuracy is selected for final testing. Training/validation accuracy, error rate, and loss curves are saved for every model.

4. Results

Because the split is approximately class-balanced, top-1 accuracy is used as the primary metric, while per-class recall and confusion matrices are used to inspect class-specific errors. Table 2 and Figure 1 report the performance of all six methods. `Net3` trained for 100 epochs is the best model with 0.752 test accuracy. The best audio model is `Net5`, which reaches 0.720 test accuracy after 80 epochs. Figure 2 supports the error analysis by showing the per-class behaviour of the best model.

5. Discussion

5.1. Effect of Architecture

The architecture comparison shows the importance of using an appropriate inductive bias. `Net1` is the weakest model, with best test accuracy 0.366, because flattening the spectrogram destroys the local time-frequency structure. A dense network must learn relationships between distant pixels from a very high-dimensional vector, which is difficult with limited data. `Net2` improves to 0.545 because convolutional filters learn local patterns such as harmonic bands, percussive textures and energy changes. `Net3` and `Net4` then improve further by stabilising this CNN structure. `Net5` demonstrates that LSTMs are also suitable when audio is represented as a sequence, but its performance depends strongly on the quality and length of the audio features. `Net6` adds a GAN stage, so its performance depends not only on the LSTM but also on whether generated samples are realistic and label-consistent.

5.2. Effect of Batch Normalisation

Comparing `Net2` and `Net3` isolates the effect of adding batch normalisation to the CNN. `Net3` improves from 0.535–0.545 test accuracy for `Net2` to 0.703–0.752. This is a large gain, so batch normalisation is not just a minor implementation detail. It normalises intermediate activations, which improves gradient flow and makes optimisation less sensitive to parameter scale. On a small dataset such as GTZAN, this also has a mild regularising effect because it reduces unstable activation shifts during training. The improvement at 100 epochs suggests that batch normalisation helps the model keep learning useful convolutional features for longer before overfitting dominates.

5.3. Effect of Optimiser

`Net3` and `Net4` have the same architecture, so their comparison is the fairest optimiser experiment. RMSProp is competitive: `Net4` reaches 0.713 test accuracy at 50 epochs and 0.723 at 100 epochs. However, it does not outperform Adam in `Net3`, which reaches 0.752. RMSProp adapts the learning rate using a moving average of squared gradients, which can help with non-stationary gradients. Adam also uses adaptive scaling but additionally includes momentum estimates, which may explain its stronger final result here. The higher validation loss for `Net4` at 100 epochs suggests that RMSProp might require a different learning rate or stronger regularisation to match Adam.

5.4. Effect of Training Length

The 50/100 epoch comparison shows that longer training is useful only when validation behaviour also improves. `Net1`, `Net3`, and `Net4` obtain higher test accuracy at 100 epochs, suggesting that these models were still benefiting

Model	Input	Main architecture	Optimiser	LR	Batch	Epochs
Net1	Spectrogram	FC: 512, 128	Adam	10^{-4}	16	50/100
Net2	Spectrogram	Conv-Conv-Pool-Conv-Conv-Pool-FC	Adam	10^{-4}	16	50/100
Net3	Spectrogram	Net2 + BatchNorm	Adam	10^{-4}	16	50/100
Net4	Spectrogram	Same as Net3	RMSProp	10^{-4}	16	50/100
Net5	Audio features	2-layer BiLSTM + FC	Adam	10^{-4}	16	80
Net6	Audio features	Conditional GAN + Net5	Adam	10^{-4}	16	80

Table 1. Summary of the six required models and training settings.

Model	Epochs	Val Acc	Test Acc	Val Loss	Test Loss	Notes
Net1	50	0.352	0.317	1.837	1.911	image
Net1	100	0.417	0.366	1.741	1.880	image
Net2	50	0.543	0.545	1.381	1.293	image
Net2	100	0.538	0.535	1.411	1.309	image
Net3	50	0.673	0.703	0.995	0.944	image
Net3	100	0.729	0.752	0.912	0.805	image
Net4	50	0.688	0.713	1.275	1.051	image
Net4	100	0.693	0.723	1.805	1.520	image
Net5	80	0.765	0.720	1.176	1.310	audio
Net6	80	0.725	0.680	1.035	1.084	audio + GAN

Table 2. Validation and test results for all six required models.

Comparison	Δ Test Acc.	Finding
Net2 100 $\rightarrow$ Net3 100	+0.217	BatchNorm helps most
Net3 100 $\rightarrow$ Net4 100	-0.030	Adam better here
Net3 50 $\rightarrow$ Net3 100	+0.049	longer BN-CNN training helps
Net2 50 $\rightarrow$ Net2 100	-0.010	slight overfitting
Net5 $\rightarrow$ Net6	-0.040	GAN samples not beneficial

Table 3. Ablation-style summary of the main experimental factors.

from additional optimisation. In contrast, `Net2` decreases slightly from 0.545 to 0.535, which indicates that longer training can also overfit. The saved curves show the typical pattern: training accuracy continues increasing while validation loss flattens or rises. For `Net5`, training accuracy reaches almost 1.0 while validation accuracy peaks earlier, again showing that the best checkpoint should be chosen by validation accuracy rather than the final epoch.

5.5. Effect of GAN Augmentation

`Net6` does not improve over `Net5`: test accuracy decreases from 0.720 to 0.680. This does not mean the idea of GAN augmentation is wrong; rather, it shows that generated data quality is critical. The GAN only augments the training set, so validation and test data remain clean. However, if generated feature sequences are unrealistic, mode-collapsed, or weakly matched to their genre labels, they can act like noisy training examples. In that case, doubling the training set does not necessarily increase useful diversity. A stronger generator, better sample filtering, or spectrogram/audio-domain augmentation might be needed before GAN augmentation improves the LSTM.

5.6. Error Analysis

The confusion reports show that not all genres are equally separable. For the best CNN, metal has recall 1.00, while rock has recall only 0.30. This suggests that high-energy spectral patterns are easy to detect for metal, but rock overlaps with metal, country and blues in instrumentation and timbre. For the LSTM model, jazz and metal are strong, but hiphop and rock have recall 0.50 and reggae has recall 0.60. These weaker classes likely share rhythmic or production characteristics with disco, pop or rock. More generally, pairs such as rock/metal, disco/pop, blues/country and classical/jazz are difficult because they share tempo, instrumentation or spectral texture. This explains why overall accuracy is limited even when useful representations are learned.

6. Conclusion

All six required classifiers were implemented and evaluated. The best model is `Net3`, a batch-normalised CNN trained for 100 epochs, with 75.2% test accuracy. The best audio model is `Net5`, a bidirectional LSTM on 3-second audio feature sequences, with 72.0% test accuracy. The results show that CNNs are preferable to dense networks for spectrogram images, batch normalisation is beneficial, and GAN augmentation helps when synthetic samples are realistic and discriminative. A limitation is that the experiments use one fixed split, so rankings may vary with another random seed. Future work should repeat evaluation over multiple seeds, test stronger or pretrained audio representations, and apply quality filtering to GAN-generated samples.

7. Reflection: Generative Models for Audio Data Augmentation in Music Classification

Generative audio augmentation is important because many audio classification tasks suffer from limited labelled data. This topic connects to the first four lab discussions: Lab 1 discussed data augmentation and model invariance, Labs 2 and 3 introduced generative models including VAEs, GANs and diffusion models, and Lab 4 discussed temporal modelling with LSTM-style sequence models. In this coursework, GTZAN contains only a small number of tracks per genre, while genres such as rock, blues, country and metal share instrumentation, rhythm and spectral texture. This makes deep classifiers prone to overfitting and genre confusion. Generative models are relevant because they aim to learn the data distribution and create additional training examples, rather than only transforming existing samples.

The main technology I used is a conditional GAN. A GAN contains a generator, which creates synthetic samples from noise and a class condition, and a discriminator, which tries to distinguish real samples from generated ones. In my `Net6` experiment, the conditional GAN generates genre-conditioned audio feature sequences and adds them only to the training set. This directly matches the coursework requirement of augmenting the LSTM audio classifier with GAN-generated audio samples. Other current technologies include DCGAN-style spectrogram generators for sound-classification augmentation [1], WaveGAN for raw waveform generation, SpecGAN for spectrogram generation, variational autoencoders, self-supervised audio representation models, and diffusion-based audio generators.

Diffusion models are especially relevant as a modern alternative to GANs. A diffusion model corrupts real data with Gaussian noise in a fixed forward process, then learns a reverse denoising process that maps noise back to realistic data. Recent IEEE/ACM work such as Diff-Sound shows how diffusion can be applied to text-to-sound generation through tokenised Mel-spectrogram representations [2]. For this coursework, a genre-conditioned diffusion model could generate extra Mel-spectrograms or audio feature sequences for each genre. Diffusion models are not one of my implemented classifiers, but they are strongly connected to the same augmentation problem as `Net6`: learning the audio distribution and generating useful training data.

At my current implementation level, I can implement a basic CNN classifier, an LSTM classifier and a simple conditional GAN in PyTorch. I understand how to keep validation and test sets clean, so that synthetic samples only affect training and do not leak into evaluation. I also understand the basic diffusion idea: sample a timestep, add noise according to a schedule, train a network to predict the noise, and repeatedly denoise during generation. However, I cannot yet reliably implement high-quality raw audio generation. The main difficulties are waveform dimensionality, unstable GAN training, mode collapse, audio architecture design, diffusion noise schedules, and evaluating whether generated samples are realistic and label-consistent. My `Net6` result shows this clearly: GAN augmentation reduced test accuracy compared with the original LSTM, suggesting that synthetic data can harm performance if it is noisy or not genre-specific.

The positive impact of generative augmentation is that it can reduce data collection costs, improve class balance, and support low-resource applications such as music retrieval, environmental sound recognition, medical audio analysis and accessibility tools. It can also make models more robust by increasing training diversity. However, poor-quality generated data can introduce bias, noise or artificial patterns into the classifier. Generated music also raises copyright and artistic-style imitation concerns, especially if models are trained on protected recordings. Realistic audio generation can also be misused to create deceptive content.

My future vision is that audio augmentation should combine stronger generative models with strict quality control. GANs are useful but difficult to train, while diffusion models may offer more stable generation at the cost of slower sampling. For future music genre classification systems, I would use conditional diffusion or improved GANs to generate class-balanced audio or Mel-spectrogram samples, then filter them using validation performance, human listening tests, and similarity checks against real data. Synthetic data should only be used when it demonstrably improves generalisation. Overall, generative augmentation is valuable not because it always improves accuracy, but because it provides a principled way to address data scarcity when generated samples are realistic, diverse and correctly labelled.

References

[1] B. Bahmei, E. Birmingham, and S. Arzanpour. CNN-RNN and Data Augmentation Using Deep Convolutional Generative Adversarial Network for Environmental Sound Classification. *IEEE Signal Processing Letters*, vol. 29, pp. 682-686, 2022, doi: 10.1109/LSP.2022.3150258.

[2] D. Yang, J. Yu, H. Wang, W. Wang, C. Weng, Y. Zou, and D. Yu. Diffsound: Discrete diffusion model for text-to-sound generation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 31:1720–1733, 2023.